Practice 07 — A Private Database (Amazon RDS)

Objective: run a PostgreSQL database that **only the app tier can reach**, connect to it from the app tier, and understand how a snapshot restore gives you a **new** database. These are the skills from today's lecture.

Timebox: in class: ~55–60 min on Track A, ~40 min on Track B. If you've been stuck on one step for more than 10 minutes, stop and ask. Don't push through alone.

> **How you're checked:** this practice is done in class and checked **orally**. There is no report. When you finish, keep your terminal (and console, on Track A) open. The instructor will ask you to show the items in the *Definition of done* and to explain what you did.

Pick your track

You don't need AWS access to pass. Pick **one** track:

	Track A — real AWS	**Track B — no AWS**
Needs	AWS Academy Learner Lab (or your own account), ~15 min of waiting for RDS	Docker (or Podman) and a bash shell (Linux, macOS, or WSL / Git Bash on Windows)
Database	RDS for PostgreSQL, `db.t3.micro`	`postgres:17` container
Slow step	Creating the DB instance: **start it first**	none

The ideas are the same on both tracks:

AWS (Track A)	Track B equivalent
Private DB subnets (no route to the internet)	`docker network create --internal db-private`
DB subnet group (2 AZs)	No equivalent: one machine, one "AZ"
SG-DB: 5432 only from SG-APP	Only containers on `db-private` can resolve and reach the DB
Public access: **No**	**No** `-p 5432:5432`: nothing is published on your laptop
RDS instance	`course-db` container
Endpoint DNS name	Container name `course-db` (Docker's built-in DNS)
Generated master password (Secrets Manager in production)	A generated password in an env file you never type
App server in SG-APP	A throwaway `app` container on `db-private` running `psql`
EBS storage behind RDS	Named volume `course-db-data`
Manual snapshot	`pg_dump -Fc` file
Restore → **new** instance, **new** endpoint	New container `course-db-restored`

Definition of done

Be ready to **show** each of these:

- □ The database is running and is **not reachable from your laptop** (Track A: `nc` times out and the endpoint resolves to a private IP. Track B: no published port, and the name doesn't resolve outside `db-private`).
- □ The rule that lets the app tier in (Track A: SG-DB inbound 5432 with **source = SG-APP**. Track B: the `db-private` network).
- □ A connection from the app tier (Track A: over TLS, `\conninfo` shows SSL) and a `products` table with at least 3 rows.
- □ `app_user` can `SELECT` and `INSERT`, and `DELETE` is denied.
- □ Track B: a `snapshot.dump` restored into a **new** database with a **different endpoint** that has your rows. (Track A: snapshot and restore are a stretch; be ready to explain what a restore creates.)
- □ Track B: the data survived removing and recreating the container.
- □ Cleanup done and shown.

Track A — real AWS (AWS Academy Learner Lab)

> **Using AWS Academy Learner Lab**
>
> 1. Open the **AWS Academy Learner Lab** course → **Modules** → **Launch AWS Academy Learner Lab** → **Start Lab**. Wait until the dot next to **AWS** turns green (1–3 min), then click **AWS** to open the console.
> 2. Use region `us-east-1` (N. Virginia). Per the Learner Lab Readme, only `us-east-1` and `us-west-2` are allowed; other regions fail.
> 3. CLI: use the terminal on the lab page (the AWS CLI is already set up), or copy the credentials from **AWS Details** → **AWS CLI** → **Show** into `~/.aws/credentials` on your laptop. They change every session.
> 4. Key pair: don't create one. Use the lab's `vockey` : **AWS Details** → **Download PEM** saves `labsuser.pem` .
> 5. When you're done, clean up (A7) and click **End Lab**.

Creating the DB takes **5–15 minutes**, so do A1–A3 first, start A3, and do A4 while it's being created. Budget about **55–60 minutes** for this track.

A1. Network and security groups

The **default VPC** is the fastest choice for class. Its subnets are technically public, but the DB will still be private: it gets **no public IP** (`--no-publicly-accessible`) and SG-DB only trusts SG-APP. In a real project, use the private DB subnets from the week 4 VPC (set `VPC` and `SUBNETS` by hand instead).

```
export AWS_REGION=us-east-1

VPC=$(aws ec2 describe-vpcs --filters Name=is-default,Values=true \
  --query 'Vpcs[0].VpcId' --output text)

SG_APP=$(aws ec2 create-security-group --group-name lab07-sg-app \
  --description "lab07 app tier" --vpc-id $VPC --query GroupId --output text)
SG_DB=$(aws ec2 create-security-group --group-name lab07-sg-db \
  --description "lab07 db tier" --vpc-id $VPC --query GroupId --output text)

# SSH to the app server from your IP only
MYIP=$(curl -s https://checkip.amazonaws.com)
aws ec2 authorize-security-group-ingress --group-id $SG_APP \
  --protocol tcp --port 22 --cidr $MYIP/32

# PostgreSQL to the DB from the app tier's SECURITY GROUP, not from an IP range
aws ec2 authorize-security-group-ingress --group-id $SG_DB \
  --protocol tcp --port 5432 --source-group $SG_APP
```

A2. DB subnet group (two AZs)

```
SUBNETS=$(aws ec2 describe-subnets --filters Name=vpc-id,Values=$VPC \
  --query 'Subnets[0:2].SubnetId' --output text)
aws ec2 describe-subnets --subnet-ids $SUBNETS \
  --query 'Subnets[].[SubnetId,AvailabilityZone]' --output table   # must be 2 different AZs

aws rds create-db-subnet-group --db-subnet-group-name lab07-db-subnets \
  --db-subnet-group-description "lab07 DB subnets" --subnet-ids $SUBNETS
```

A3. Create the database (start it now)

Generate the master password on your laptop and keep it in a variable, so you never type it or paste it into a file:

```
PW=$(openssl rand -base64 18 | tr -d '/@" ')
echo "$PW" > .dbpw && chmod 600 .dbpw     # only so you can read it again later; never commit it

aws rds create-db-instance \
  --db-instance-identifier lab07-db \
  --engine postgres \
  --db-instance-class db.t3.micro \
  --allocated-storage 20 --storage-type gp3 \
  --master-username dbadmin --master-user-password "$PW" \
  --db-name shop \
  --db-subnet-group-name lab07-db-subnets \
  --vpc-security-group-ids $SG_DB \
  --no-publicly-accessible --no-multi-az \
  --storage-encrypted --backup-retention-period 1
```

`db.t3.micro` is the safe default in Learner Lab. `db.t4g.micro` also works if your lab allows it.

> **The production way:** instead of `--master-user-password`, use `--manage-master-user-password`. RDS then creates the password itself and stores it in **Secrets Manager**, and the app reads it through its instance profile. Try it only if your lab allows Secrets Manager.

Don't wait here. Go to A4 now and come back with:

```
aws rds wait db-instance-available --db-instance-identifier lab07-db
```

A4. Launch the app server (while the DB is being created)

```
APP_ID=$(aws ec2 run-instances \
  --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --instance-type t3.micro --key-name vockey \
  --security-group-ids $SG_APP \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=lab07-app}]' \
  --query 'Instances[0].InstanceId' --output text)
aws ec2 wait instance-running --instance-ids $APP_ID

APP_IP=$(aws ec2 describe-instances --instance-ids $APP_ID \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text)
echo $APP_IP
```

Download `labsuser.pem` (AWS Details → Download PEM) into this folder and run `chmod 400 labsuser.pem`.

> In the week 4 design the app server sits in a private subnet behind the web tier. Today it has a public IP only so you can SSH to it quickly. The **database** is the private part.

A5. Find the endpoint, prove the DB is private

On your laptop, once the DB is `available`:

```
ENDPOINT=$(aws rds describe-db-instances --db-instance-identifier lab07-db \
  --query 'DBInstances[0].Endpoint.Address' --output text)
echo $ENDPOINT

nslookup $ENDPOINT           # → a PRIVATE address (172.31.x.x in the default VPC)
nc -vz -w 5 $ENDPOINT 5432   # → times out. That's the point.

cat .dbpw                    # the password you generated in A3; you'll paste it once
```

In a real app, nobody copies the password by hand: it lives in Secrets Manager and the server reads it through its **instance profile** (lecture 6).

A6. Connect from the app tier over TLS

```
ssh -i labsuser.pem ec2-user@$APP_IP
```

On the app server:

```
sudo dnf install -y postgresql17 || sudo dnf install -y postgresql16 || sudo dnf install -y
postgresql15
curl -sO https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem

export ENDPOINT=<paste the endpoint>
psql "host=$ENDPOINT port=5432 dbname=shop user=dbadmin sslmode=verify-full sslrootcert=global-
bundle.pem"
# paste the password when asked
```

Inside `psql`:

```
\conninfo                     -- must mention SSL

CREATE TABLE products (
  id    serial PRIMARY KEY,
  name  text NOT NULL,
  price numeric(10,2) NOT NULL
);
INSERT INTO products (name, price) VALUES
  ('Keyboard', 25.00), ('Mouse', 12.50), ('Monitor', 180.00);

-- least privilege: the app can read and add products, nothing else
CREATE ROLE app_user LOGIN PASSWORD 'change-me-app';
GRANT SELECT, INSERT ON products TO app_user;
GRANT USAGE ON SEQUENCE products_id_seq TO app_user;
\q
```

Now test `app_user` (predict each result first):

```
psql "host=$ENDPOINT dbname=shop user=app_user password=change-me-app sslmode=require" \
  -c "INSERT INTO products (name, price) VALUES ('Webcam', 40.00);" \
  -c "SELECT * FROM products;" \
  -c "DELETE FROM products;"      # → permission denied
```

A7. Cleanup (required, because RDS bills per hour)

```
aws rds delete-db-instance --db-instance-identifier lab07-db \
  --skip-final-snapshot --delete-automated-backups
aws ec2 terminate-instances --instance-ids $APP_ID

aws rds wait db-instance-deleted --db-instance-identifier lab07-db
aws rds delete-db-subnet-group --db-subnet-group-name lab07-db-subnets

aws ec2 wait instance-terminated --instance-ids $APP_ID
aws ec2 delete-security-group --group-id $SG_DB     # SG_DB first: it references SG_APP
aws ec2 delete-security-group --group-id $SG_APP
rm -f .dbpw
```

If you did the stretch below, also delete `lab07-db-restored` and the snapshot `lab07-db-snap`. Show `aws rds describe-db-instances` with nothing `lab07` left, then click **End Lab**.

Stretch (optional, only with 15+ minutes left): snapshot and restore

Before cleanup:

```
aws rds create-db-snapshot --db-instance-identifier lab07-db \
  --db-snapshot-identifier lab07-db-snap
aws rds wait db-snapshot-available --db-snapshot-identifier lab07-db-snap

aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier lab07-db-restored \
  --db-snapshot-identifier lab07-db-snap \
  --db-instance-class db.t3.micro \
  --db-subnet-group-name lab07-db-subnets \
  --vpc-security-group-ids $SG_DB \
  --no-publicly-accessible
aws rds wait db-instance-available --db-instance-identifier lab07-db-restored

aws rds describe-db-instances \
  --query 'DBInstances[?starts_with(DBInstanceIdentifier,`lab07`)].
[DBInstanceIdentifier,Endpoint.Address]' \
  --output table             # two different endpoints
```

Each step takes as long as creating the DB, so this rarely fits in class. Either way, be ready to explain what a restore creates and what you'd have to change in the app.

Track B — no AWS (Docker)

Every command works with `podman` instead of `docker`. Run everything in one folder, e.g. `mkdir lab07 && cd lab07`.

B1. The "private subnet" and the storage

```
docker network create --internal db-private   # --internal: no route to the internet
docker volume create course-db-data
```

B2. A password you never type (the Secrets Manager idea)

```
PW=$(openssl rand -hex 16)
printf 'POSTGRES_USER=dbadmin\nPOSTGRES_PASSWORD=%s\n' "$PW" > db.env
printf 'PGHOST=course-db\nPGPORT=5432\nPGUSER=dbadmin\nPGPASSWORD=%s\nPGDATABASE=postgres\n' "$PW"
> app.env
unset PW
chmod 600 db.env app.env
```

`db.env` configures the database, and `app.env` is what the app gets: the endpoint (`PGHOST`) and the credentials. Neither file ever goes into Git.

B3. Start the database: private, no published port

```
docker run -d --name course-db --network db-private --env-file db.env \
  -v course-db-data:/var/lib/postgresql/data postgres:17

# wait until it accepts connections over the network
until docker run --rm --network db-private --env-file app.env postgres:17 pg_isready -q; do sleep
1; done
```

There is **no** `-p 5432:5432`. That's today's "Public access: No".

B4. Connect from the "app tier" and create data

Save as `schema.sql`:

```
CREATE TABLE products (
  id    serial PRIMARY KEY,
  name  text NOT NULL,
  price numeric(10,2) NOT NULL
);
INSERT INTO products (name, price) VALUES
  ('Keyboard', 25.00), ('Mouse', 12.50), ('Monitor', 180.00);

-- least privilege: the app can read and add products, nothing else
CREATE ROLE app_user LOGIN PASSWORD 'change-me-app';
GRANT SELECT, INSERT ON products TO app_user;
GRANT USAGE ON SEQUENCE products_id_seq TO app_user;
```

```
# a throwaway "app server" container on the same network, reaching the DB by NAME
docker run --rm -i --network db-private --env-file app.env postgres:17 \
  psql -v ON_ERROR_STOP=1 < schema.sql
docker run --rm --network db-private --env-file app.env postgres:17 \
  psql -c 'SELECT * FROM products;'
```

Want an interactive shell instead? `docker run --rm -it --network db-private --env-file app.env postgres:17 psql`

Test `app_user` (predict each result first):

```
docker run --rm --network db-private --env-file app.env \
  -e PGUSER=app_user -e PGPASSWORD=change-me-app postgres:17 \
  psql -c "INSERT INTO products (name, price) VALUES ('Webcam', 40.00);" \
       -c 'SELECT count(*) FROM products;' \
       -c 'DELETE FROM products;'          # → permission denied for table products
```

B5. Prove the database is private

```
docker port course-db               # prints nothing: no published ports
nc -vz -w 3 localhost 5432          # → connection refused (your laptop can't reach it)

# a container that is NOT on db-private (like a server outside the VPC)
docker run --rm postgres:17 pg_isready -h course-db -t 5   # → no response: the name doesn't
resolve
```

> Honest limit: Docker network isolation is weaker than a security group. On some setups (OrbStack, Docker Desktop) a container's **IP** can still be reached from your laptop or another network. The hard

> guarantee here is that nothing is published on the host. Real security groups block by default, at every IP.

B6. Storage outlives the container

```
docker rm -f course-db                        # "the instance is gone"
docker run -d --name course-db --network db-private --env-file db.env \
  -v course-db-data:/var/lib/postgresql/data postgres:17
until docker run --rm --network db-private --env-file app.env postgres:17 pg_isready -q; do sleep
1; done
docker run --rm --network db-private --env-file app.env postgres:17 \
  psql -c 'SELECT count(*) FROM products;'    # still 4 rows
```

B7. Snapshot → disaster → restore into a NEW database

```
# manual snapshot
docker exec course-db pg_dump -U dbadmin -d postgres -Fc > snapshot.dump
ls -l snapshot.dump

# the disaster
docker run --rm --network db-private --env-file app.env postgres:17 \
  psql -c 'DROP TABLE products;'

# restore = a NEW instance with a NEW "endpoint"
docker run -d --name course-db-restored --network db-private --env-file db.env \
  -v course-db-restored-data:/var/lib/postgresql/data postgres:17
until docker run --rm --network db-private --env-file app.env \
  -e PGHOST=course-db-restored postgres:17 pg_isready -q; do sleep 1; done

docker exec -i course-db-restored pg_restore -U dbadmin -d postgres \
  --no-owner --no-privileges < snapshot.dump

# the app has to be pointed at the new endpoint
docker run --rm --network db-private --env-file app.env \
  -e PGHOST=course-db-restored postgres:17 psql -c 'SELECT * FROM products;'
```

Now try logging in as `app_user` on `course-db-restored`. It fails, because `pg_dump` saves one database, **not** the server's roles. An RDS snapshot is a copy of the whole instance, so its users come back too. Be ready to explain the difference.

Stretch (discussion only): a read replica would be a second PostgreSQL server that streams changes from `course-db`. Setting up streaming replication is too much for today. Instead, be ready to explain how a read replica differs from Multi-AZ (lecture, Block 6).

B8. Cleanup

```
docker rm -f course-db course-db-restored
docker volume rm course-db-data course-db-restored-data
docker network rm db-private
rm -f db.env app.env snapshot.dump schema.sql
```

Be ready to explain

1. Why is SG-DB's source **SG-APP** instead of an IP range like `10.0.2.0/24`? (Track B: what plays that role on your machine?)
2. Your database is single-AZ. Why does the DB subnet group still need subnets in **two** AZs?
3. How did you create the master password, and why didn't you type it or put it in a file you'd commit? How would a real app get it?
4. When you restore a snapshot, what changes, and what must you update in the app?
5. Your laptop can't connect to the database. Bug or feature? How would a developer legitimately get in?
6. Manual snapshot vs automated backups with point-in-time restore: when would you use each?

Common mistakes

- `UnauthorizedOperation` **/ "not authorized" in Learner Lab (Track A).** You're in the wrong region (use `us-east-1`), the lab session expired (click **Start Lab** again and refresh the credentials), or you used a feature the lab blocks (e.g. Secrets Manager): use the plain `--master-user-password` path.
- **Waiting for RDS and doing nothing.** Creation takes 5–15 minutes. Start A3, then do A4 while it runs.
- `psql` **from the laptop "doesn't work" (Track A).** That's correct: the DB is private. Connect from the app server.
- **Connection times out from the app server too (Track A).** SG-DB's inbound rule has the wrong source (an IP or the wrong group), or the app instance isn't in SG-APP.
- `create-db-subnet-group` **fails with an AZ error (Track A).** Both subnets are in the same AZ. Pick subnets from two different AZs.
- `sslmode=verify-full` **fails (Track A).** You didn't download `global-bundle.pem`, or you used an IP instead of the endpoint name. The certificate is issued for the name.
- `DependencyViolation` **deleting a security group (Track A).** Delete SG-DB before SG-APP (SG-DB's rule references SG-APP), and wait until the DB and the EC2 instance are fully deleted.
- `could not translate host name "course-db"` **from the app container (Track B).** You forgot `--network db-private` on the `docker run`.
- **"Connection refused" right after starting (Track B).** PostgreSQL is still initialising. Use the `pg_isready` loop.
- **Data is gone after B6 (Track B).** You recreated the container without `-v course-db-data:/var/lib/postgresql/data`, or you removed the volume.

- **Forgot cleanup (Track A).** A forgotten RDS instance bills every hour, and a stopped one starts again by itself after 7 days. Delete it.